详细设计报告

1、系统完整功能描述

1、 描述

在一块固定大小的区域内，游戏玩家通过控制贪吃蛇的移动去吃食物，吃到食物后蛇身体长度加 1，分数加 1。食物被蛇吃到后立马消失，并再次随机产生。蛇撞到四周墙壁或者自己身体时死亡。

2、 实现目标

1.蛇的移动、蛇的死亡；

2.吃食物、产生食物。

3、按键约定

为提升用户的体验感，我们对操作按键做以下约定：

1.空格键/enter 键控制游戏开始；

2.上下左右按键分别控制贪吃蛇的四个运动方向。

4、实现原理

1.利用 canvas 画布完成运动场地、食物、贪吃蛇的展示；

2.利用数组存储贪吃蛇的坐标位置；

3.利用上、下、左、右键改变贪吃蛇的蛇头坐标；

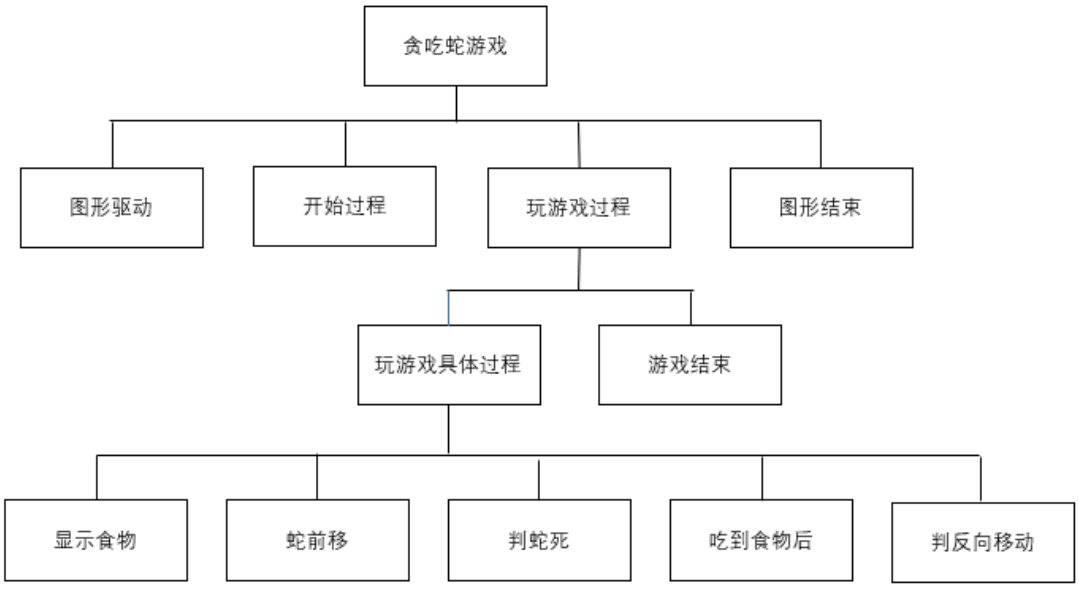
4.不断重新绘制页面，给人造成贪吃蛇运动的错觉。

2、各模块流程等说明

模块应具有高内聚性、低耦合性。这样程序运行才稳定，效率更高。

根据功能将贪吃蛇游戏划分为四个大模块，七个小模块。每个模块均有自己的功能，而且，比较容易画出各模块的流程图。

模块枝干图



简要说明：图形驱动即完成初始化图形系统功能；开始画面模块的主要作用是根据设置绘制出玩游戏区域四周的墙壁；玩游戏的过程是此游戏程序的核心，又可细分为好多小的功能模块；其中，显示食物模块仅完成显示食物功能；蛇前移模块即通过算法实现蛇的某方向移动；判蛇死模块的功能是通过判断蛇是否碰到墙壁或自己来判断蛇是否死亡。吃到食物后模块只有蛇吃到食物后才起作用。即当蛇吃到食物后，蛇自身延长一节并绘出蛇。通过键盘操控来玩游戏主要由判反向移动模块完成，即当蛇不是反向移动时，按照键盘上方向键指示移动。游戏结束模块功能简单，仅完成提示语功能与输出分数。最后图形结束模块完成关闭图形功能。

2.1 代码截图+实现功能（如：食物判断模块、控制蛇的移动模块）

2.1.1 图形驱动模块

```
/* 定义事件执行监测 */
function onKeyDown(event) {
    if (status == -1) {
        status = 0;
        food();
        run();
    }
    if (window.event) // IE
    {
        keynum = event.keyCode;
    } else if (event.which) // Netscape/Firefox/Opera
    {
        keynum = event.which;
    }
    if (keynum == 38 && head_for != 0)
        head_for = 3;
    if (keynum == 40 && head_for != 3)
        head_for = 0;
    if (keynum == 37 && head_for != 2)
        head_for = 1;
    if (keynum == 39 && head_for != 1)
        head_for = 2;
    if (keynum == 80)
        pause_flag = !pause_flag;
    if (keynum != 80)
        pause_flag = false;
}
```

2.1.2 开始画面模块

```
/* 页面变量初始化 */
function Init() {
    width = document.getElementById('canvas3d').clientWidth;
    height = document.getElementById('canvas3d').clientHeight;
    renderer = new THREE.WebGLRenderer({
        antialias: true
    });
    renderer.setSize(width, height);
    document.getElementById('canvas3d').innerHTML = '';
    document.getElementById('canvas3d').appendChild(renderer.domElement);
    renderer.setClearColor(Math.random() * 0xffffff, 1.0);
    camera = new THREE.PerspectiveCamera(45, width / height, 1, 5000);
    camera.position.x = 0;
    camera.position.y = -280;
    camera.position.z = 200;
    camera.up.x = 0;
    camera.up.y = 0;
    camera.up.z = 1;
    camera.lookAt({
        x: 0,
        y: 0,
        z: 0
    });
    scene = new THREE.Scene(); /**  * 创建场景对象Scene*/

    light = new THREE.DirectionalLight('white', 1.0, 0);
    light.position.set(-600, -600, -600);
    scene.add(light);
    plane = CreatePlane(400);
    plane.position.set(-5, -5, -5);
    scene.add(plane);
    //start_scene
    start_scene = new THREE.Scene();
    word = CreateText('snake 3D', 80, 40);
    word2 = CreateText('press any key to start', 30, 20);
    start_scene.add(word);
    start_scene.add(word2);
    start_scene.add(light);
    renderer.render(start_scene, camera);
    for (i = 0; i < nx; i++) {
        board[i] = new Array()
        for (k = 0; k < ny; k++) {
            board[i][k] = 0;
        }
    } //0 = none, 1 = snake body, 2 = food
    fo = CreateCube(10, 10, 50);
    scene.add(fo);
    document.addEventListener('keydown', onKeyDown, false);
    for (i = 0; i < len; i++) {
        snake[i] = new Object();
        snake[i].x = head_pos_x + i * dir_x[3 - head_for];
        snake[i].y = head_pos_y + i * dir_y[3 - head_for];
        cube[i] = CreateCube(10, 10, 10);
        cube[i].position.x = snake[i].x * 10 - 200;
        cube[i].position.y = -snake[i].y * 10 + 190;
        scene.add(cube[i]);
        board[snake[i].x][snake[i].y] = 1;
    }

}
```

2.1.3 显示食物模块

```
/* 定义食物的坐标 */
function food() {
    var tx, ty;
    do {
        tx = Math.ceil(Math.random() * 1000) % nx;
        ty = Math.ceil(Math.random() * 1000) % ny;
    } while (board[tx][ty]);
    board[tx][ty] = 2;
    fo.position.x = tx * 10 - 200;
    fo.position.y = -ty * 10 + 190;
    fo.position.z = 20;
}
```

2.1.4 蛇向前移动模块

```
/* 定义蛇的上下左右移动 */
function move() {
    var tx = snake[0].x + dir_x[head_for];
    var ty = snake[0].y + dir_y[head_for];
    //tx = (tx + nx) % nx;
    //ty = (ty + ny) % ny;
    if (tx >= 0 && tx < nx && ty >= 0 && ty < ny) {
        if (board[tx][ty] != 1) {
            the_last_head = head_for;
            if (board[tx][ty] == 2) {
                snake[len] = new Object();
                snake[len].x = snake[len - 1].x;
                snake[len].y = snake[len - 1].y;
                cube[len] = CreateCube(10, 10, 10);
                cube[len].position.x = snake[len].x * 10 - 200;
                cube[len].position.y = -snake[len].y * 10 + 190;
                scene.add(cube[len]);
                board[tx][ty] = 1;
                len++;
                food();
            }
            for (i = len - 1; i > 0; i--) {
                snake[i].x = snake[i - 1].x;
                snake[i].y = snake[i - 1].y;
            }
            snake[0].x = tx;
            snake[0].y = ty;
        } else {
            if (the_last_head + head_for != 3) {
                alert("game over!\ryour score is " + len);
                reload_game();
            } else {
                head_for = the_last_head;
            }
        }
    } else {
        alert("game over!\ryour score is " + len);
        reload_game();
    }
    for (i = 0; i < nx; i++) {
        for (k = 0; k < ny; k++) {
            if (board[i][k] == 1)
                board[i][k] = 0;
        }
    }
    for (i = 0; i < len; i++) {
        board[snake[i].x][snake[i].y] = 1;
    }
}
```

2.1.5 判蛇死模块

```
/* 定义蛇的上下左右移动 */
function move() {
    var tx = snake[0].x + dir_x[head_for];
    var ty = snake[0].y + dir_y[head_for];
    //tx = (tx + nx) % nx;
    //ty = (ty + ny) % ny;
    if (tx >= 0 && tx < nx && ty >= 0 && ty < ny) {
        if (board[tx][ty] != 1) {
            the_last_head = head_for;
            if (board[tx][ty] == 2) {
                snake[len] = new Object();
                snake[len].x = snake[len - 1].x;
                snake[len].y = snake[len - 1].y;
                cube[len] = CreateCube(10, 10, 10);
                cube[len].position.x = snake[len].x * 10 - 200;
                cube[len].position.y = -snake[len].y * 10 + 190;
                scene.add(cube[len]);
                board[tx][ty] = 1;
                len++;
                food();
            }
            for (i = len - 1; i > 0; i--) {
                snake[i].x = snake[i - 1].x;
                snake[i].y = snake[i - 1].y;
            }
            snake[0].x = tx;
            snake[0].y = ty;
        } else {
            if (the_last_head + head_for != 3) {
                alert("game over!\ryour score is " + len);
                reload_game();
            } else {
                head_for = the_last_head;
            }
        }
    } else {
        alert("game over!\ryour score is " + len);
        reload_game();
    }
    for (i = 0; i < nx; i++) {
        for (k = 0; k < ny; k++) {
            if (board[i][k] == 1)
                board[i][k] = 0;
        }
    }
    for (i = 0; i < len; i++) {
        board[snake[i].x][snake[i].y] = 1;
    }
}
```

2.1.6 吃到食物后处理模块

```
/* 定义蛇的上下左右移动 */
function move() {
    var tx = snake[0].x + dir_x[head_for];
    var ty = snake[0].y + dir_y[head_for];
    //tx = (tx + nx) % nx;
    //ty = (ty + ny) % ny;
    if (tx >= 0 && tx < nx && ty >= 0 && ty < ny) {
        if (board[tx][ty] != 1) {
            the_last_head = head_for;
            if (board[tx][ty] == 2) {
                snake[len] = new Object();
                snake[len].x = snake[len - 1].x;
                snake[len].y = snake[len - 1].y;
                cube[len] = CreateCube(10, 10, 10);
                cube[len].position.x = snake[len].x * 10 - 200;
                cube[len].position.y = -snake[len].y * 10 + 190;
                scene.add(cube[len]);
                board[tx][ty] = 1;
                len++;
                food();
            }
            for (i = len - 1; i > 0; i--) {
                snake[i].x = snake[i - 1].x;
                snake[i].y = snake[i - 1].y;
            }
            snake[0].x = tx;
            snake[0].y = ty;
        } else {
            if (the_last_head + head_for != 3) {
                alert("game over!\ryour score is " + len);
                reload_game();
            } else {
                head_for = the_last_head;
            }
        }
    } else {
        alert("game over!\ryour score is " + len);
        reload_game();
    }
    for (i = 0; i < nx; i++) {
        for (k = 0; k < ny; k++) {
            if (board[i][k] == 1)
                board[i][k] = 0;
        }
    }
    for (i = 0; i < len; i++) {
        board[snake[i].x][snake[i].y] = 1;
    }
}
```

2.1.7 盘蛇反向移动模块

```
/* 定义蛇的前移或后移 */
function run() {
    if (status == -1)
        return;
    if (!pause_flag)
        move();
    render();
    setTimeout("run()", 100); //you can change speed here
}
```

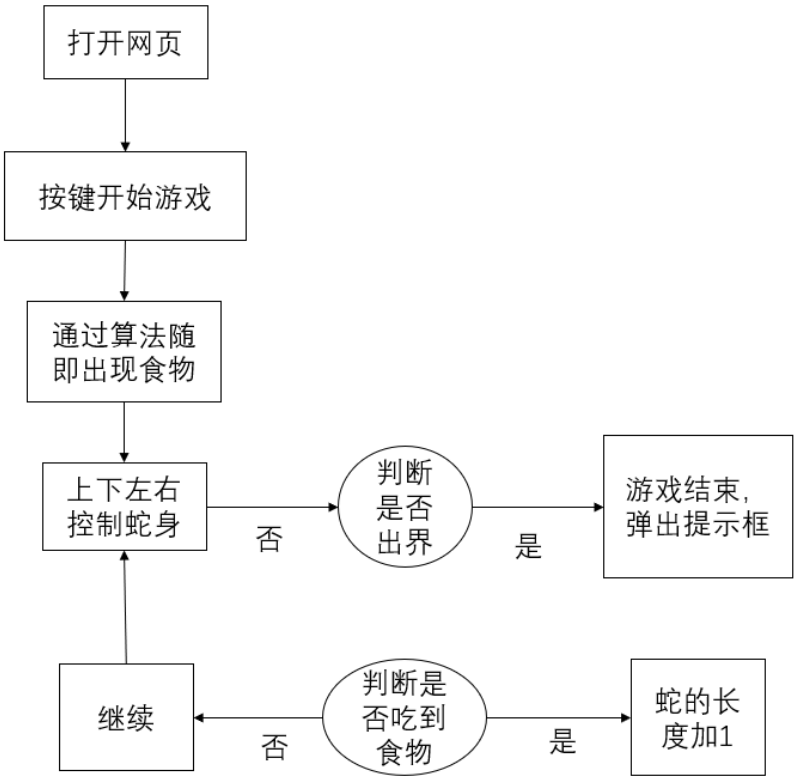
2.1.8 游戏结束模块

```
/* 定义蛇的上下左右移动 */
function move() {
    var tx = snake[0].x + dir_x[head_for];
    var ty = snake[0].y + dir_y[head_for];
    //tx = (tx + nx) % nx;
    //ty = (ty + ny) % ny;
    if (tx >= 0 && tx < nx && ty >= 0 && ty < ny) {
        if (board[tx][ty] != 1) {
            the_last_head = head_for;
            if (board[tx][ty] == 2) {
                snake[len] = new Object();
                snake[len].x = snake[len - 1].x;
                snake[len].y = snake[len - 1].y;
                cube[len] = CreateCube(10, 10, 10);
                cube[len].position.x = snake[len].x * 10 - 200;
                cube[len].position.y = -snake[len].y * 10 + 190;
                scene.add(cube[len]);
                board[tx][ty] = 1;
                len++;
                food();
            }
            for (i = len - 1; i > 0; i--) {
                snake[i].x = snake[i - 1].x;
                snake[i].y = snake[i - 1].y;
            }
            snake[0].x = tx;
            snake[0].y = ty;
        } else {
            if (the_last_head + head_for != 3) {
                alert("game over!\ryour score is " + len);
                reload_game();
            } else {
                head_for = the_last_head;
            }
        }
    } else {
        alert("game over!\ryour score is " + len);
        reload_game();
    }
    for (i = 0; i < nx; i++) {
        for (k = 0; k < ny; k++) {
            if (board[i][k] == 1)
                board[i][k] = 0;
        }
    }
    for (i = 0; i < len; i++) {
        board[snake[i].x][snake[i].y] = 1;
    }
}
```

2.1.9 图形结束

```
/* 定义蛇的上下左右移动 */
function move() {
    var tx = snake[0].x + dir_x[head_for];
    var ty = snake[0].y + dir_y[head_for];
    //tx = (tx + nx) % nx;
    //ty = (ty + ny) % ny;
    if (tx >= 0 && tx < nx && ty >= 0 && ty < ny) {
        if (board[tx][ty] != 1) {
            the_last_head = head_for;
            if (board[tx][ty] == 2) {
                snake[len] = new Object();
                snake[len].x = snake[len - 1].x;
                snake[len].y = snake[len - 1].y;
                cube[len] = CreateCube(10, 10, 10);
                cube[len].position.x = snake[len].x * 10 - 200;
                cube[len].position.y = -snake[len].y * 10 + 190;
                scene.add(cube[len]);
                board[tx][ty] = 1;
                len++;
                food();
            }
            for (i = len - 1; i > 0; i--) {
                snake[i].x = snake[i - 1].x;
                snake[i].y = snake[i - 1].y;
            }
            snake[0].x = tx;
            snake[0].y = ty;
        } else {
            if (the_last_head + head_for != 3) {
                alert("game over!\ryour score is " + len);
                reload_game();
            } else {
                head_for = the_last_head;
            }
        }
    } else {
        alert("game over!\ryour score is " + len);
        reload_game();
    }
    for (i = 0; i < nx; i++) {
        for (k = 0; k < ny; k++) {
            if (board[i][k] == 1)
                board[i][k] = 0;
        }
    }
    for (i = 0; i < len; i++) {
        board[snake[i].x][snake[i].y] = 1;
    }
}
```

2.2 算法流程



3、实现工具、开发环境、工具库

1、必备技能：

1.HTML（主要是 div 盒子模型，canvas 画布）

2.CSS（为你好看的游戏界面做准备）

3.JavaScript（让小蛇动起来，逻辑代码实现）

2、开发工具&开发环境：

1.HBuilder X，轻巧方便。

2.Google 浏览器

3、工具库：

Jquery、Three.js、optimer_bold.typeface.js。

4、进度

详细设计

11 月 18 日之前完成详细设计，提交详细设计报告，要求内容如下：

1.系统完整功能描述（10.21~11.17）

2.各模块流程、结构具体实现，关键函数、变量等说明（10.21~11.17）

3.所采用实现工具，开发环境，主要工具库等（10.21~10.24）

4.目前进度

最终设计

12 月 15 日之前完成最终设计，提交内容包括:
最终课程设计报告，包含内容如下：
1.系统实现功能详细描述（详细设计再详细）（11.18~12.11）
2.核心功能源文件清单，核心代码说明（11.18~12.11）
3.实现功能展示(截图或视频)（12.11）
4.小组成员实际完成工作情况及自评（12.11）
课程设计展示汇报材料，包括报告用幻灯片，展示用视频或图片（12.12~12.14）